EE-205 : Machine Learning for Signal Processing

Homework # 2

AMRIT LAL (24608)

M.Tech - SP

Dept. - ECE

Q.01.) One-class SVM

Sol.) given $X = \{x_1, x_2 \dots x_\ell\} \in \mathbb{R}^n$

$$\|\phi(x_j) - a\|^2 \le R^2 + \xi_j$$

$R \rightarrow$ radius, $a \rightarrow$ center } of smallest sphere

$\phi(x) \rightarrow$ non-linear feature space

$\xi_j \rightarrow$ slack variable $(j = 1, 2, \dots, \ell)$

$$R^2 + C \sum_{j=1}^{\ell} \xi_j \qquad C \rightarrow \text{penalty term}$$

a.) give the primal form Lagrangian & the primal constraints for the one-class SVM.

Sol.) Objective function ; $\underset{a, R, \xi}{\arg\min} \; R^2 + C \sum_{j=1}^{\ell} \xi_j$

primal constraints

i) $\|\phi(x_j) - a\|^2 \le R^2 + \xi_j \quad \forall \; j = 1, 2, \dots, \ell$

ii) $\xi_j \ge 0$

So, lagrangian function,

$$L(R, a, \xi, \alpha, \mu) = R^2 + C \sum_{j=1}^{\ell} \xi_j - \sum_{j=1}^{\ell} \alpha_j \xi_j + \sum \mu_j \left[(\phi(x_j) - a)^T (\phi(x_j) - a) - R^2 - \xi_j \right]$$

----- (1)

b.)

Sol.) Constraints :

i) $\|\phi(x_j) - a\|^2 \le R^2 + \xi_j \quad \forall \; j = 1, 2, \dots, \ell$

ii) $\xi_j \geq 0$

iii) $\alpha_j \geq 0$ & $\mu_j \geq 0$ → KKT constraint

iv) $\alpha_j \xi_j = 0$

v) $\mu_j \left[\|\phi(x_j) - a\|^2 - R^2 - \xi_j \right] = 0$ } → complementary slackness

- Support vectors are those vectors (x_j) which satisfy the equality $\|\phi(x_j) - a\| = R + \xi_j$
- The support vectors will change when $C > 1$ is choosen. the constant C indirectly determines which vector must be support vectors as follows.

A large value of C means mis-classification is heavily penalized which will make most of the ξ_j's $\longrightarrow 0$. this is possible only if most of the points are inside the sphere.

if $C \rightarrow$ small $\longrightarrow$ not heavily penalized

So, R is made to be as small as possible.

As $\xi_j \geq 0$ so, all points on or outside the decision sphere are support vectors. hence, as $C \uparrow$ no. of support vectors decreases & vice versa.

differentiation of eq^n ① w.r.t. R, a & ξ_j

$$\frac{\partial L}{\partial R} = 2R + \sum_{j=1}^{l} \mu_j (-2R) = 0$$

$$\Rightarrow \sum_{j=1}^{l} \mu_j = 1 \quad \text{---} \quad (2)$$

$$\frac{\partial L}{\partial a} = \frac{\partial}{\partial a} \sum_{j=1}^{l} \mu_j \left[a^T a - 2\phi^T(x_j)\, a \right] = \sum_{j=1}^{l} \left(2a - 2\phi(x_j) \right) \mu_j = 0$$

$$\Rightarrow 2a \sum_{j=1}^{l} \mu_j = 2 \sum_{j=1}^{l} \mu_j \phi(x_j)$$

$$\therefore \sum_{j} \mu_j = 1$$

$$\Rightarrow a = \sum_{j=1}^{l} \mu_j \phi(x_j) \quad \text{- - - -} \quad (3)$$

hence $\underline{a}$ is a weighted avg. of all the data points in the transformed feature space.

$$\frac{\partial L}{\partial \xi_j} = C - \alpha_j - \mu_j = 0 \quad \Rightarrow \quad \alpha_j + \mu_j = C \quad \cdots\cdots (4)$$

by eq^n (1), (2), (3) and (4)

$$L(R, a, \xi, \alpha, \mu) = R^2 + C\sum_{j=1}^{l}\xi_j - \sum_{j=1}^{l}\alpha_j\xi_j + \sum_{l}\mu_j\left[(\phi(x_j)-a)^T(\phi(x_j)-a) - R^2 - \xi_j\right]$$

by $\sum_l \mu_j = 1$ & $\alpha_j + \mu_j = C$

$$\Rightarrow L(R, a, \xi, \alpha, \mu) = \sum_{j=1}^{l}\mu_j\left[\phi^T(\underline{x}_j)\,\phi(\underline{x}_j) - 2\underline{a}^T\phi(\underline{x}_j) + \underline{a}^T\underline{a}\right]$$

$$\therefore \underline{a} = \sum_l \mu_j \phi(\underline{x}_j)$$

$$= \sum_{j=1}^{l}\mu_j\left[\phi^T(\underline{x}_j)\,\phi(\underline{x}_j) - a^Ta\right]$$

$$= \sum_{j=1}^{l}\mu_j\,\phi^T(\underline{x}_j)\,\phi(\underline{x}_j) - \underbrace{\left(\sum_{j=1}^{l}\mu_j\right)}_{:=1}\underline{a}^T\underline{a}$$

$$K(x_j, x_j) = \phi^T(\underline{x}_j)\,\phi(\underline{x}_j) \longrightarrow \text{kernal function}$$

$$\therefore a^Ta = \left(\sum_l \mu_k\,\phi^T(x_k)\right)\left(\sum_l \mu_j\,\phi(x_j)\right) = \sum_l\sum_l \mu_j\mu_k\,K(x_j, x_k)$$

So,

$$L(R, a, \xi, \alpha, \mu) = \sum_{j=1}^{l}\mu_j\,K(x_j, x_j) - \sum_{j=1}^{l}\sum_{k=1}^{l}\mu_j\mu_k\,K(x_j, x_k)$$

$$= g(\mu) \longrightarrow \text{dual form in terms of kernel function}$$

we need to maximize $g(\mu)$ w.r.t. μ_j for $j = 1, 2, \cdots, l$

$$\frac{\partial g}{\partial \mu_j} = K(x_j, x_j) - 2 \sum_{k=1}^{l} \mu_k K(x_j, x_k) = 0$$

$$\Rightarrow \quad K(x_j, x_j) = 2 \sum_{k=1}^{l} \mu_k K(x_j, x_k) \quad \Big\}_{j=1}^{l}$$

matrix form ;

$$2 \begin{bmatrix} K(x_1,x_1) & K(x_1,x_2) & \cdots & K(x_1,x_l) \\ K(x_2,x_1) & K(x_2,x_2) & \cdots & K(x_2,x_l) \\ \vdots & \vdots & & \\ K(x_l,x_1) & K(x_l,x_2) & \cdots & K(x_l,x_l) \end{bmatrix} \begin{bmatrix} \mu_1 \\ \mu_2 \\ \\ \mu_l \end{bmatrix} = \begin{bmatrix} K(x_1,x_1) \\ K(x_2,x_2) \\ \vdots \\ K(x_l,x_l) \end{bmatrix}$$

where we need to solve for $\mu_1, \mu_2, \cdots, \mu_l$

$\therefore \|\phi(x_j) - a\|^2 = R^2$; where x_j is not only a support vector but also lies on the decision boundary.

$\alpha_j \xi_j = 0 \longrightarrow$ points lying on / inside $\longrightarrow \xi_j = 0$

$\mu_j \left[\|\phi(x_j) - a\|^2 - R^2 - \xi_j \right] = 0 \longrightarrow$ points lying on sphere

for on sphere $\longrightarrow \mu_j > 0$

as $\alpha_j + \mu_j = C$ & $\mu_j > 0$ so $\alpha_j < C$ for points lying on the sphere

for the set of all support vectors on the sphere

$$0 < \alpha_j < C \quad \& \quad \mu_j > 0$$

using above condition,

$R^2 = \|\phi(x_j) - a\|^2$ for $x_j \in S$

where $S = \{ x_j \; ; \; 0 < \alpha_j < C \quad \& \quad \mu_j > 0 \}$

though this relation holds for any point $x_j \in S$ the solution obtained from this scheme is not numerically stable.

If N_s is no. of element in S then

$$R^2 = \frac{1}{N_s} \sum_{s,j} \|\phi(x_j) - a\|^2$$

to get a numerically stable solution

c.)

Soln for new data points $\underline{x}$

the optimal value of $R^2 = \frac{1}{N_s} \sum_{s,j} \|\phi(x_j) - a\|^2$ and $a = \sum_{j=1}^{l} \mu_j \phi(x_j)$

for a point $\underline{x}$ to be predicted as an outlier, we compute

$$\|\phi(x) - a\|^2 \rightarrow \begin{cases} < R^2 \rightarrow \text{not outliers} \\ > R^2 \rightarrow \text{outliers} \end{cases}$$

Now,

$$R^2 = \frac{1}{N_s} \sum_{s,j} (\phi(x_j) - a)^T (\phi(x_j) - a)$$

$$= \frac{1}{N_s} \sum_{s,j} K(x_j, x_j) - \frac{2}{N_s} \sum_{s,j} \phi^T(x_j)\, a + a^T a$$

$$= \frac{1}{N_s} \sum_{s,j} K(x_j, x_j) - \frac{2}{N_s} \sum_{s,j} \sum_{i=1}^{l} \mu_i K(x_j, x_j) + \sum_{i=1}^{l} \sum_{j=1}^{l} \mu_i \mu_j K(x_j, x_j)$$

$$\therefore \|\phi(x) - a\|^2 = (\phi(x) - a)^T (\phi(x) - a) = K(x,x) - 2\phi^T(x)\, a + a^T a$$

$$= K(x,x) - 2 \sum_{i=1}^{l} \mu_i K(x_i, x) + a^T a$$

$$\therefore \underbrace{\|\phi(x) - a\|^2 - R^2}_{\psi(x)} = K(x,x) - 2 \sum \mu_i K(x_i, x) - \frac{1}{N_s} \sum_{s,j} K(x_j, x_j) + \frac{2}{N_s} \sum_{s,j} \sum_{i=1}^{l} \mu_i K(x_i, x_j)$$

$$\Rightarrow \psi(x) = \left[K(x,x) - \frac{1}{N_s} \sum_{s,j} K(x_j, x_j) \right] - 2 \sum_{i=1}^{l} \mu_i \left[K(x_i, x) - \frac{1}{N_s} \sum_{s,j} K(x_i, x_j) \right]$$

if $\psi(x) > 0$, we say it is an outlier

and if $\psi(x) < 0$ then we say it is not an outlier.

Q. 04) Neural Networks - Cost function :

define NN with softmax

$\{O_i\}_{i=1}^{M}$ → i/p

$\{y_i\}_M$ → target

$y \in B^{C \times 1}$

M → no. of data points

B → boolen variable (0 or 1)

C → no. of classes

data point O_i associated with only one class label C_i

$C_i \in \{1, 2, \dots, C\}$

$V_i = \{V_i^1, V_i^2 \dots, V_i^C\} \in \mathbb{R}^{C \times 1}$

$0 < V_i < 1$ $\quad$ V_i → o/p of network

NN cost function

$$J_{MSE} = \sum_{i=1}^{M} \| V_i - y_i \|^2$$

where M : no. of data points

$V_i = [V_i^1 \dots V_i^C]^T$: network o/p prob. for C classes

$y_i = [y_i^1 \dots y_i^C]^T$: one-hot encoded target vector for C classes

C_i : true class label for i^{th} data points

for one-hot encoding

$y_i^{C_i} = 1$ & $y_i^j = 0$ for $j \neq C_i$

Sol^n

$$\Rightarrow \underset{\sim}{V_i} = \frac{1}{\phi} \begin{bmatrix} z_i^1 \\ z_i^2 \\ \vdots \\ z_i^{C_i} \\ \vdots \\ z_i^C \end{bmatrix}$$

where $z_i^j = e^{w_j^T \sigma_i + b_i} \Big\}_{i=1}^{C}$

$$\phi = \sum_{q=1}^{C} e^{w_q^T \sigma_i + b_q}$$

$$\underset{\sim}{y_i} = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ 0 \\ \vdots \\ 0 \end{bmatrix} \leftarrow C_i^{th} \text{ location}$$

$$\underline{v_i} - \underline{y_i} = \begin{bmatrix} z_{i/s}^{1} \\ \vdots \\ (z_{i/s}^{c_i} - 1) \\ \vdots \\ z_{i/s}^{c} \end{bmatrix} = \frac{1}{\alpha} \begin{bmatrix} z_i^1 \\ z_i^2 \\ \vdots \\ (z_i^{c_i} - 1) \\ \vdots \\ z_i^c \end{bmatrix}$$

$$\Rightarrow \|v_i - y_i\|^2 = \frac{1}{\alpha^2} \sum_{j=1}^{c} (z_i^j)^2 + 1 - \frac{2 z_i^{c_i}}{\alpha}$$

but $\frac{z_i^{c_i}}{\alpha} = v_i^{c_i}$

$$\Rightarrow \|v_i - y_i\|^2 = \frac{1}{\alpha^2} \sum_{j=1}^{c} (z_i^j)^2 + 1 - 2 v_i^{c_i} \quad ----(1)$$

$$\therefore [1 - v_i^{c_i}]^2 = 1 - 2 v_i^{c_i} + (v_i^{c_i})^2$$

$$= 1 - 2 v_i^{c_i} + \frac{1}{\alpha^2} (z_i^{c_i})^2 \quad --- (2)$$

since $(z_i^{c_i})^2 \le \sum_{j=1}^{c} (z_i^j)^2$ by defination of z_i

$$(1 - v_i^{c_i})^2 \le \|v_i - y_i\|^2 \quad --- (A)$$

$$2(1 - v_i^{c_i})^2 = 2(1 - 2 v_i^{c_i}) + 2 (v_i^{c_i})^2$$

$$\therefore 2(1 - v_i^{c_i})^2 = 1 - 2 v_i^{c_i} + [1 - 2 v_i^{c_i} + 2 (v_i^{c_i})^2]$$

$$= 1 - 2 v_i^{c_i} + [(1 - v_i^{c_i})^2 + (v_i^{c_i})^2] \quad --- (3)$$

$$\Rightarrow (1 - v_i^{c_i})^2 = \left[1 - \frac{z_i^{c_i}}{\alpha}\right]^2 = \frac{1}{\alpha^2} [\alpha - z_i^{c_i}]^2$$

$$\therefore \alpha = z_i^1 + z_i^2 + \cdots + z_i^c \quad \text{by defination}$$

$$(\alpha - z_i^{c_i})^2 = (z_i^1 + z_i^2 + \cdots \text{(all terms except } c_i) \cdots + z_i^c)^2$$

$$\ge (z_i^1)^2 + (z_i^2)^2 + \cdots \text{all terms except } c_i \cdots + (z_i^c)^2$$

$$\Rightarrow (1 - v_i^{c_i})^2 \geq (v_i^1)^2 + (v_i^2)^2 + \cdots + (v_i^C)^2$$

all terms except c_i

$$\Rightarrow (v_i^{c_i})^2 + (1 - v_i^{c_i})^2 \geq (v_i^1)^2 + (v_i^2)^2 + \cdots + (v_i^{c_i})^2 + \cdots + (v_i^C)^2 \quad \text{---- (4)}$$

by eqn (1), (2), (3) & (4)

$$\|v_i - y_i\|^2 = 1 - 2v_i^{c_i} + [(v_i^1)^2 + \cdots + (v_i^C)^2]$$

$$\therefore 2(1 - v_i^{c_i})^2 = 1 - 2v_i^{c_i} + [(1 - v_i^{c_i})^2 + (v_i^{c_i})^2]$$

so,

$$\|v_i - y_i\|^2 \leq 2(1 - v_i^{c_i})^2 \quad \text{---} \quad (B)$$

from (A) & (B) for any i^{th} data point

$$(1 - v_i^{c_i})^2 \leq \|v_i - y_i\|^2 \leq 2(1 - v_i^{c_i})^2$$

this must also be true when summed over all data points

$$\sum_{i=1}^{M} (1 - v_i^{c_i})^2 \leq \sum_{i=1}^{M} \|v_i - y_i\|^2 \leq \sum_{i=1}^{M} 2(1 - v_i^{c_i})^2$$

Q.05.) Learning of NN weights

given, quadratic error

$$J = J_0 + \frac{1}{2}(w - w^*)^T H (w - w^*)$$

where, $w^* \rightarrow$ min^m of function

$H \rightarrow$ Hessian matrix

λ_j & $u_j \rightarrow$ Evals & Evecs of H $\quad \forall\ j = 1, 2, \ldots n$

gradient based update

$$w^{t+1} = w^t - \eta \frac{\partial J}{\partial w} \qquad t \rightarrow \text{iteration}$$

$w^0 = 0$

Sol.^{n}

$$\frac{\partial J}{\partial w} = H(w - w^*)$$

Let $w = a_1 u_1 + a_2 u_2 + \cdots + a_n u_n$

we also assume $\|u_j\| = 1 \quad \forall j = 1, \ldots, n$

$$\Rightarrow w^T u_1 = a_1 u_1^T u_1 + \cdots + a_n u_n^T u_1$$

$H \rightarrow$ hassian matrix, it must be symmetric. hence all Evecs of H must be orthogonal.

$$w^T u_1 = a_1$$

similarly for $j = 2, \ldots, n \qquad a_j = w^T u_j$

$$\therefore w = \sum_{j=1}^{n} a_j u_j$$

similarly

$$w^* = \sum_{j=1}^{n} b_j u_j \quad ; \quad b_j = w^{*T} u_j$$

$$\Rightarrow w - w^* = \sum_{j=1}^{n} (a_j - b_j) u_j$$

$$\Rightarrow H(w - w^*) = \sum_{j=1}^{n} (a_j - b_j) H u_j = \sum_{j=1}^{n} (a_j - b_j) \lambda_j u_j$$

then $H(w^t - w^*) = \sum_{j=1}^{n} (a_j^t - b_j) \lambda_j u_j$ where w^t & a_j^t are the respective values after t^{th} iteration

$$\therefore w^{t+1} = w^t - \eta \left.\frac{\partial J}{\partial w}\right|_{w^t}$$

$$= w^t - \sum_{j=1}^{n} \eta (a_j^t - b_j) \lambda_j u_j$$

$$= \sum_{j=1}^{n} a_j^t u_j - \sum_{j=1}^{n} \eta a_j^t \lambda_j u_j + \sum_{j=1}^{n} \eta b_j \lambda_j u_j$$

$$= \sum_{j=1}^{n} a_j^t (1 - \eta \lambda_j) u_j + \sum_{j=1}^{n} \eta b_j \lambda_j u_j$$

$$\Rightarrow (w^{t+1})^T u_q = a_q^t (1 - \eta \lambda_q) + \eta b_q \lambda_q \qquad \text{as } u_q^T u_j = \begin{cases} 0 ; & q \neq j \\ 1 ; & q = j \end{cases}$$

$$w_i^{t+1} = a_i^t (1-\eta\lambda_i) + \eta b_i \lambda_i = a_i^t (1-\eta\lambda_i) + \eta w^{*T} u_i \lambda_i$$
$$= a_i^t (1-\eta\lambda_i) + \eta w_i^* \lambda_i$$
$$\therefore w_i^* = w^{*T} u_i$$

$$\Rightarrow \quad a_i^t = (w^t)^T u_i = w_i^t$$
$$\therefore \quad w_i^{t+1} = w_i^t (1-\eta\lambda_i) + \eta w_i^* \lambda_i$$

$$\Rightarrow \quad w_i^1 = w_i^0 (1-\eta\lambda_i) + \eta w_i^* \lambda_i$$
$$\therefore w_i^0 = 0$$
$$w_i^1 = \eta w_i^* \lambda_i$$

$$\Rightarrow \quad w_i^2 = w_i^1 (1-\eta\lambda_i) + \eta w_i^* \lambda_i = \eta w_i^* \lambda_i - \eta^2 w_i^* \lambda_i^2 + \eta w_i^* \lambda_i$$
$$= 2\eta w_i^* \lambda_i - \eta^2 w_i^* \lambda_i^2$$
$$= [1 - (1-\eta\lambda_i)^2] w_i^*$$

Let $w_j^q = [1-(1-\eta\lambda_j)^q] w_j^*$

Also,

$$w_j^{q+1} = w_j^q (1-\eta\lambda_j) + \eta w_j^* \lambda_j = [1-(1-\eta\lambda_j)^q] w_j^* (1-\eta\lambda_j) + \eta w_j^* \lambda_j$$
$$= w_j^* - (1-\eta\lambda_j)^{q+1} w_j^*$$

$$\Rightarrow \quad w_j^{q+1} = [1-(1-\eta\lambda_j)^{q+1}] w_j^*$$

by principle of mathamatical induction for all iterate indices q

$$w^q = [1-(1-\eta\lambda_j)^q] w_j^*$$

further if $|1-\eta\lambda_i| < 1$ then $(1-\eta\lambda_j)^q \to 0$ if $q \to \infty$

$$\therefore w_j^q = w_j^*$$

$$\Rightarrow \quad (w^T)^q u_j = (w^*)^T u_j$$

for all indices j which satisfy $|1-\lambda_j\eta| < 1$. hence j^{th} component of w^q in the eigen basis.

In perticular, if all the Evals satisfy this condition, then each component of w^q approches the respective component of w^* as $q \to \infty$ in the ϵ-basis $\quad w^q \to w^*$ as $q \to \infty$

```
from google.colab import drive
drive.mount('/content/gdrive')
# List of study material in /content/gdrive to prompt for Security reasons, can't drive.mount('/content/gdrive', force_remount=True).
!ls "/content/gdrive/My Drive/Colab Notebooks/Classification"
# Load Libraries
import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
# Load libraries
import pandas
import numpy
import matplotlib
import seaborn as sns
from pandas.plotting import scatter_matrix
from sklearn.model_selection import StratifiedKFold
from sklearn.model_selection import cross_val_score
```

QUESTION 2

```
# Training_data_path = "/content/gdrive/My Drive/Colab Notebooks/Classification/"
Training_data_path = "/content/gdrive/My Drive/Colab Notebooks/Classification/"

def run(img_data_directory,img_type):
    image_information = []
    i = 1
    for filename in os.listdir(img_data_directory):
        if filename.endswith(".jpg") or filename.endswith(".tif"):
            image = cv2.imread(
                img_data_directory + filename,
                cv2.IMREAD_GRAYSCALE)
            image_information.append(image.flatten())
    return image_information

X_training, y_training = run(img_data_directory,img_data_path)
print("Training Images Done - X_training shape:", X_training.shape)

def run(img_data_directory,img_type,segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    return image_information, filenames

X_training = run(img_data_directory,img_data_path,"Otsu")
print("Training Images Done - X_training shape:", X_training.shape)

X_training = run(img_data_directory,img_data_path,"Watershed")
print("Training Images Done - X_training shape:", X_training.shape)

def run(img_data_directory,img_type,segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    return filenames

y_training = run(img_data_directory,img_data_path,"Otsu")
print("Training Labels Done - y_training shape:", y_training.shape)

y_training = run(img_data_directory,img_data_path,"Watershed")
print("Training Labels Done - y_training shape:", y_training.shape)

import numpy as np
avg_components = [5, 5, 10, 15, 20]
variance = [0.90, 0.95, 0.90, 0.90, 0.90]
tolerance_values = [0.1, 0.2, 0.05]
kernel_types = ['linear', 'rbf', 'sigmoid']

import numpy as np

# Training through different parameter combinations
for avg_components in avg_components:
    for variance in variance: # Only for PCA components
        # Split into training and validation sets
        X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)
        X_train_pca = do_pca(X_training, n_components=avg_components)

        for kernel_type in kernel_types:
            for tolerance in tolerance_values:
                # Create SVM model
                svm_model = SVC(kernel=kernel_type, C=tolerance, tol=tolerance)
                svm_model.fit(X_train_pca, y_train)
                predictions = svm_model.predict(X_val_pca)
                accuracy = accuracy_score(y_val, predictions)

                print("Parameters:")
                print("Average Components: ", avg_components)
                print("Kernel Type: ", kernel_type)
                print("Tolerance: ", tolerance)
                print("Accuracy: ", accuracy)
                print()

Processing with 5 PCA components
Processing with 5 PCA components
Processing with 10 PCA components
Processing with 15 PCA components
Processing with 20 PCA components

df_submission = pd.read_csv("/content/gdrive/My Drive/Colab Notebooks/Classification/Otsu_segmentation/Otsu_segmentation_results.csv")
df_submission.head()

print("Results saved as /content/gdrive/My Drive/Colab Notebooks/Classification/Otsu_segmentation/Otsu_segmentation_results.csv")

df_submission = pd.read_csv("/content/gdrive/My Drive/Colab Notebooks/Classification/Watershed_segmentation/Watershed_segmentation_results.csv")
df_submission.head()

print("Results saved as /content/gdrive/My Drive/Colab Notebooks/Classification/Watershed_segmentation/Watershed_segmentation_results.csv")

import/export results.zip
!zip -r /content/results.zip /content/results

from google.colab import files
files.download("/content/results.zip")

# Import/Export Results.zip file
import os
import shutil
import zipfile
shutil.rmtree('/content/results', ignore_errors=True)
os.makedirs('/content/results', exist_ok=True)
shutil.copytree('/content/gdrive/My Drive/Colab Notebooks/Classification/Otsu_segmentation', '/content/results/Otsu_segmentation')
shutil.copytree('/content/gdrive/My Drive/Colab Notebooks/Classification/Watershed_segmentation', '/content/results/Watershed_segmentation')
with zipfile.ZipFile('/content/results.zip', 'w', zipfile.ZIP_DEFLATED) as zipf:
    for root, dirs, files in os.walk('/content/results'):
        for file in files:
            zipf.write(os.path.join(root, file))
print("Results.zip file created successfully!")

# Import/Export Results.zip file
import os
import shutil
import zipfile
shutil.rmtree('/content/results', ignore_errors=True)
os.makedirs('/content/results', exist_ok=True)
shutil.copytree('/content/gdrive/My Drive/Colab Notebooks/Classification/Otsu_segmentation', '/content/results/Otsu_segmentation')
shutil.copytree('/content/gdrive/My Drive/Colab Notebooks/Classification/Watershed_segmentation', '/content/results/Watershed_segmentation')
with zipfile.ZipFile('/content/results.zip', 'w', zipfile.ZIP_DEFLATED) as zipf:
    for root, dirs, files in os.walk('/content/results'):
        for file in files:
            zipf.write(os.path.join(root, file))
print("Results.zip file created successfully!")

from google.colab import files
files.download("/content/results.zip")
```

QUESTION 3

```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 4

```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 5

```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 6

```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 7

```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 8

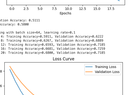
```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 9

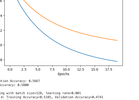
```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_information + "Otsu_segmentation/"
    elif segmentation_type == "Watershed":
        image_information = image_information + "Watershed_segmentation/"
    filenames = os.listdir(image_information)
    image_information = [cv2.imread(image_information + filename, cv2.IMREAD_GRAYSCALE).flatten() for filename in filenames]
    # Train Logistic Regression model
    model = LogisticRegression(max_iter=1000)
    model.fit(X_training, y_training)
    # Predict on the training data
    predictions = model.predict(image_information)
    return predictions, filenames

predictions, filenames = run(X_training, y_training, img_data_directory, img_type, segmentation_type)
print("Predictions:", predictions)
print("Filenames:", filenames)

# Create a DataFrame with filenames and predictions
df = pd.DataFrame({'Filename': filenames, 'Prediction': predictions})
df.to_csv('predictions.csv', index=False)
print("Predictions saved to predictions.csv")

import matplotlib.pyplot as plt
import numpy as np
from sklearn.metrics import accuracy_score
from sklearn.linear_model import LogisticRegression

# Assuming X_training, y_training, img_data_directory, img_type, segmentation_type are defined

# Split the data into training and validation sets
X_train, X_val, y_train, y_val = train_test_split(X_training, y_training, test_size=0.2, random_state=42)

# Initialize lists to store accuracies
accuracies = []

# Train Logistic Regression model and calculate accuracy for each epoch
for epoch in range(1, 11):  # Assuming 10 epochs
    model = LogisticRegression(max_iter=epoch)
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)
    accuracy = accuracy_score(y_val, predictions)
    accuracies.append(accuracy)

# Plot the accuracy curve
plt.plot(range(1, 11), accuracies)
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.title('Accuracy Curve')
plt.show()

Training with batch size=32, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=32, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.1
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.01
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
Training with batch size=64, learning rate=0.001
Epoch 10: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 20: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 30: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 40: Training Accuracy=0.000, Validation Accuracy=0.000
Epoch 50: Training Accuracy=0.000, Validation Accuracy=0.000
```

QUESTION 10

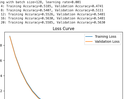
```
from sklearn.linear_model import LogisticRegression
def run(X_training, y_training, img_data_directory, img_type, segmentation_type):
    image_information = img_data_directory, img_type
    if segmentation_type == "Otsu":
        image_information = image_
```



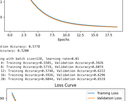

```
Validation Accuracy: 0.5111
Test Accuracy: 0.5000
```



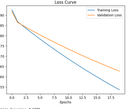

```
Training with batch size=128, learning rate=0.001
Epoch 0: Training Accuracy=0.4397, Validation Accuracy=0.4711
Epoch 5: Training Accuracy=0.6407, Validation Accuracy=0.5111
Epoch 10: Training Accuracy=0.6875, Validation Accuracy=0.5000
Epoch 15: Training Accuracy=0.6938, Validation Accuracy=0.5481
Epoch 20: Training Accuracy=0.7281, Validation Accuracy=0.5638

Validation Accuracy: 0.5667
Test Accuracy: 0.5500
```




```
Training with batch size=128, learning rate=0.01
Epoch 0: Training Accuracy=0.4444, Validation Accuracy=0.5026
Epoch 5: Training Accuracy=0.5733, Validation Accuracy=0.5074
Epoch 10: Training Accuracy=0.6125, Validation Accuracy=0.5185
Epoch 15: Training Accuracy=0.6526, Validation Accuracy=0.5444
Epoch 20: Training Accuracy=0.6844, Validation Accuracy=0.5519

Validation Accuracy: 0.5778
Test Accuracy: 0.5100
```



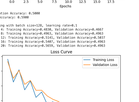

```
Training with batch size=128, learning rate=0.1
Epoch 0: Training Accuracy=0.4667, Validation Accuracy=0.4667
Epoch 5: Training Accuracy=0.4938, Validation Accuracy=0.4963
Epoch 10: Training Accuracy=0.5000, Validation Accuracy=0.4963
Epoch 15: Training Accuracy=0.4938, Validation Accuracy=0.4963
Epoch 20: Training Accuracy=0.5062, Validation Accuracy=0.4963

Validation Accuracy: 0.5556
Test Accuracy: 0.5200
```



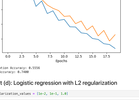
Part (d): Logistic regression with L2 regularization

In [ ]:
```
regularization_values = [0.1, 1.0]

for reg_value in regularization_values:
    print(f"Training with L2 regularization (lambda={reg_value}):")
    model = LogisticRegression(params[0], reg=True, lambda=reg_value, total_iterations=20)
    model.train_on_batch(x_train, batch_size=128)

    val_predictions = model.predict(x_val)
    val_accuracy = np.mean(val_predictions == y_val)

    test_predictions = model.predict(test_x_pca)
    test_accuracy = model.get_accuracy(test_predictions, test_b)

    results_summary.append({
        "Regularization Coefficient": reg_value,
        "Validation Accuracy": val_accuracy,
        "Test Accuracy": test_accuracy
    })

    print("Validation Accuracy: {val_accuracy:.4f}")
    print("Test Accuracy: {test_accuracy:.4f}")
```

```
Training with L2 regularization (lambda=0.1)
Epoch 0: Training Accuracy=0.4889, Validation Accuracy=0.5926
Epoch 5: Training Accuracy=0.5694, Validation Accuracy=0.5889
Epoch 10: Training Accuracy=0.6781, Validation Accuracy=0.6222
Epoch 15: Training Accuracy=0.6844, Validation Accuracy=0.6444
Epoch 20: Training Accuracy=0.6906, Validation Accuracy=0.6444
```




```
Validation Accuracy: 0.6222
Test Accuracy: 0.6000

Training with L2 regularization (lambda=0.1)
Epoch 0: Training Accuracy=0.5000, Validation Accuracy=0.5852
Epoch 5: Training Accuracy=0.5719, Validation Accuracy=0.6074
Epoch 10: Training Accuracy=0.6719, Validation Accuracy=0.6259
Epoch 15: Training Accuracy=0.6906, Validation Accuracy=0.6370
Epoch 20: Training Accuracy=0.7000, Validation Accuracy=0.6444
```




```
Validation Accuracy: 0.6111
Test Accuracy: 0.5900

Training with L2 regularization (lambda=0.01)
Epoch 0: Training Accuracy=0.4889, Validation Accuracy=0.5926
Epoch 5: Training Accuracy=0.5719, Validation Accuracy=0.6074
Epoch 10: Training Accuracy=0.6719, Validation Accuracy=0.6259
Epoch 15: Training Accuracy=0.6938, Validation Accuracy=0.6296
Epoch 20: Training Accuracy=0.6844, Validation Accuracy=0.6333
```

```
Validation Accuracy: 0.5889
Test Accuracy: 0.5900
```

Summarizing Results:

Observations on Overfitting/Underfitting

For $\lambda = 0.01$, the model overfits as the training loss is low while the validation loss increases significantly. For $\lambda = 0.1$, there is a balance between overfitting and underfitting, but performance is suboptimal. For $\lambda = 1$, the model likely underfits as both losses remain high, and there is minimal improvement in accuracy.

Performance on Test Set

The test accuracy remains relatively stable across all three regularization settings:

Test accuracy for $\lambda = 0.01$: 60% (highest)

Test accuracy for $\lambda = 0.1$: 59%

Test accuracy for $\lambda = 1$: is 59%

In conclusion, while $\lambda = 0.01$ suffers from overfitting and $\lambda = 1$ from underfitting, $\lambda = 0.1$ provides a reasonable trade-off between these issues, achieving the highest test accuracy.

QUESTION 6

In [ ]:
```
class CustomNeuralNetworkModel:
    def __init__(self, input_features, hidden_features, output_features):
        # Initialize weights and biases for input layer to hidden layer
        self.first_layer_weights = np.random.randn(input_features, hidden_features) * 0.01
        self.first_layer_bias = np.zeros((1, hidden_features))
        self.second_layer_weights = np.random.randn(hidden_features, output_features) * 0.01
        self.second_layer_bias = np.zeros((1, output_features))

    def activation_function_relu(self, x):
        return np.maximum(0, x)

    def derivative_relu(self, x):
        return x > 0.astype(float)

    def activation_function_softmax(self, x):
        exp_x = np.exp(x - np.max(x, axis=1, keepdims=True))
        return exp_x / np.sum(exp_x, axis=1, keepdims=True)

    def forward_propagation(self, input_data):
        self.hidden_layer_input = np.dot(input_data, self.first_layer_weights) + self.first_layer_bias
        self.hidden_layer_output = self.activation_function_relu(self.hidden_layer_input)

        self.output_layer_input = np.dot(self.hidden_layer_output, self.second_layer_weights) + self.second_layer_bias
        self.output_layer_output = self.activation_function_softmax(self.output_layer_input)

        return self.output_layer_output

    def backward_propagation(self, input_data, target_labels, learning_rate=0.0625):
        num_samples = input_data.shape[0]

        self.forward_propagation(input_data)

        output_layer_error = self.output_layer_output - target_labels
        hidden_layer_error = output_layer_error.dot(self.second_layer_weights.T) * self.derivative_relu(self.hidden_layer_input)

        gradient_second_layer_weights = (1 / num_samples) * np.dot(self.hidden_layer_output.T, output_error)
        gradient_second_layer_bias = (1 / num_samples) * np.sum(output_error, axis=0, keepdims=True)

        gradient_first_layer_weights = (1 / num_samples) * np.dot(input_data.T, hidden_gradient)
        gradient_first_layer_bias = (1 / num_samples) * np.sum(hidden_gradient, axis=0, keepdims=True)

        self.second_layer_weights -= learning_rate * gradient_second_layer_weights
        self.second_layer_bias -= learning_rate * gradient_second_layer_bias
        self.first_layer_weights -= learning_rate * gradient_first_layer_weights
        self.first_layer_bias -= learning_rate * gradient_first_layer_bias

        epsilon = 1e-15
        loss_value = -np.mean(np.log(self.output_layer_output[range(num_samples), target_labels] + epsilon))

        return loss_value
```

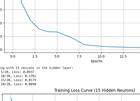
Function to train and evaluate the neural network model

In [ ]:
```
def train_and_evaluate_model(train_input_data, train_target_labels, test_input_data, test_target_labels, hidden_features_count, epochs=20):
    reduced_train_input_data, mean_train_input_data, principal_components = apply_pca_transform(train_input_data, num_components=2)
    reduced_test_input_data = do_pca_transform(test_input_data - mean_train_input_data, principal_components)

    model_instance = CustomNeuralNetworkModel(input_features=reduced_train_input_data.shape[1],
                                              hidden_features=hidden_features_count,
                                              output_features=2)

    training_losses = []

    for epoch in range(epochs):
        epoch_loss = model_instance.backward_propagation(reduced_train_input_data,
                                                         train_target_labels,
                                                         learning_rate=0.0625)

        training_losses.append(epoch_loss)

        if (epoch + 1) % 5 == 0:
            print(f"Epoch {epoch + 1}/{epochs}, Loss: {epoch_loss:.4f}")

    train_predictions = np.argmax(model_instance.forward_propagation(reduced_train_input_data), axis=1)
    test_predictions = np.argmax(model_instance.forward_propagation(reduced_test_input_data), axis=1)

    training_accuracy = accuracy_score(train_target_labels, train_predictions)
    testing_accuracy = accuracy_score(test_target_labels, test_predictions)

    return training_losses, training_accuracy, testing_accuracy
```

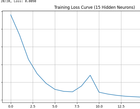
In [ ]:
```
# Test the neural network with different numbers of hidden neurons
hidden_neurons_list = [10, 15, 20, 25, 30]
performance_results = []
```

In [ ]:
```
for neurons in hidden_neurons_list:
    print(f"Training with {neurons} neurons in the hidden layer:")

    losses_recorded, train_acc_recorded, test_acc_recorded = train_and_evaluate_model(
                                                                x_train,
                                                                y_training,
                                                                x_testing,
                                                                y_testing,
                                                                neurons)

    performance_results.append({"hidden_neurons": neurons,
                                "train": losses_recorded,
                                "test_accuracy": test_acc_recorded})

    # Plot loss curve
    plt.figure(figsize=(10, 5))
    plt.plot(losses_recorded)
    plt.title(f"Training Loss Curve ({neurons} Hidden Neurons)")
    plt.xlabel("Epochs")
    plt.ylabel("Cross-Entropy Loss")
    plt.grid()
    plt.show()

    # Plot test accuracy
    plt.figure(figsize=(10, 5))
    plt.plot([result["test_accuracy"] for result in performance_results],
             [result["hidden_neurons"] for result in performance_results],
             marker='o')
    plt.title("Test Accuracy vs Hidden Layer Size")
    plt.xlabel("Number of Hidden Neurons")
    plt.ylabel("Test Accuracy")
    plt.grid()
    plt.show()
```

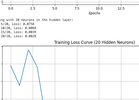
```
Training with 10 neurons in the hidden layer:
Epoch 5/20, Loss: 0.4869
Epoch 10/20, Loss: 0.4144
Epoch 15/20, Loss: 0.3844
Epoch 20/20, Loss: 0.3666
```



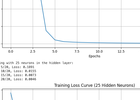

```
Training with 15 neurons in the hidden layer:
Epoch 5/20, Loss: 0.4837
Epoch 10/20, Loss: 0.4125
Epoch 15/20, Loss: 0.3775
Epoch 20/20, Loss: 0.4058
```



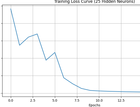

```
Training with 20 neurons in the hidden layer:
Epoch 5/20, Loss: 0.4716
Epoch 10/20, Loss: 0.3968
Epoch 15/20, Loss: 0.3664
Epoch 20/20, Loss: 0.4019
```

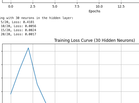



```
Training with 25 neurons in the hidden layer:
Epoch 5/20, Loss: 0.4691
Epoch 10/20, Loss: 0.3955
Epoch 15/20, Loss: 0.3653
Epoch 20/20, Loss: 0.4044
```




```
Training with 30 neurons in the hidden layer:
Epoch 5/20, Loss: 0.4583
Epoch 10/20, Loss: 0.3954
Epoch 15/20, Loss: 0.3652
Epoch 20/20, Loss: 0.4017
```





SUMMARY

Initially with 10 hidden neurons, the test accuracy is approximately 60%. As the number of neurons increases to 15 and 20, the accuracy improves significantly, reaching a peak of 100% at 20 neurons. However, increasing the number of hidden neurons further to 25 results in a sharp drop in accuracy to around 60%. Finally, with 30 hidden neurons, the test accuracy recovers and again reaches 100%.